

# Reviewer Pack — Sign-Rank Distinguishing Tensor for Read-Twice Branching Pro...

Entry d2ba6b1a27c2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 01:46:09 UTC

## Sign-Rank Distinguishing Tensor for Read-Twice Branching Programs

**Entry ID:** d2ba6b1a27c2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-29 01:46:09 UTC

### 1. Conjecture

**Field A** (mathematical branch): discrepancy theory **Field B** (complexity object): matrix rigidity

#### Statement:

For any read-twice branching program  $P$  computing a Boolean function  $f$ , the sign-rank of its communication matrix  $M_P$  is  $O(\log \text{size}(P))$ , but for the inner product mod 2 function  $IP_2$ , the sign-rank of its trivial read-twice BP is  $\Omega(n)$ .

#### Rationale (proposer’s reasoning):

This conjecture leverages the connection between sign-rank (a discrepancy-theoretic invariant) and matrix rigidity (a field\_B concept). The  $IP_2$  function’s exponential lower bound for read-once BPs suggests its read-twice representation must exhibit high sign-rank, while read-twice BPs for general functions should have logarithmic sign-rank due to their structured computation. This bridges the gap between BP complexity and communication complexity barriers.

**Taxonomy category:** DISPERSION\_DISCREPANCY (status at proposal time: alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 37f311b5acd4acf7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

default:  $\geq 80\%$  seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| NATURAL_PROOFS | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.95       | UNCERTAIN        | SAFE             |
| KARP_LIPTON    | SAFE    | 0.95       | SAFE             | UNCERTAIN        |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate entries below the pivot
        for j in range(i+1, n):
            factor = Fraction(A[j][i], A[i][i])
            for k in range(n):
                A[j][k] -= factor * A[i][k]

    return A

def sign_rank(matrix):
    n = len(matrix)
    matrix = [[Fraction(x) for x in row] for row in matrix]
    rank = 0
    for i in range(n):
        if any(A[i][j] != 0 for j in range(i, n)):
            rank += 1
    return rank

def generate_random_sign_matrix(n, rank):
    A = [[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)]
    U = gaussian_elimination(A)
    while sign_rank(U) < rank:
        A = [[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)]
        U = gaussian_elimination(A)
    return A
```

```

def generate_read_twice_bp_matrix(n):
    A = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            A[i][j] = random.choice([-1, 1])
            A[j][i] = -A[i][j]
    return A

def run_trial(seed: int) -> dict:
    random.seed(seed)

    instances_tested = 0
    total_sign_rank = 0

    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5):
            if n == 40 and _ >= 2:
                continue # Skip the last two instances for n=40 to stay under time limit
            instances_tested += 1

            if n == 40:
                A = generate_read_twice_bp_matrix(n)
            else:
                rank = math.ceil(math.log2(n))
                A = generate_random_sign_matrix(n, rank)

            sign_rank_value = sign_rank(A)
            total_sign_rank += sign_rank_value

    average_sign_rank = total_sign_rank / instances_tested
    conjecture_holds = average_sign_rank <= 10 * math.log(40) # Upper bound for n=40
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Average Sign-Rank",
        "metric_value": average_sign_rank,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_sign_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:

```

```

    print(f"RESULT: SUPPORTED mean={mean_sign_rank} std=0 support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2cd38001.py", line 100, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2cd38001.py", line 77, in run_trial
    A = generate_random_sign_matrix(n, rank)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2cd38001.py", line 47, in generate_random_sign_matrix
    U = gaussian_elimination(A)
        ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2cd38001.py", line 30, in gaussian_elimination
    factor = Fraction(A[j][i], A[i][i])
              ^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(0, 0)

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test crashed with ZeroDivisionError during Gaussian elimination, preventing data collection | next: Debug gaussian\_elimination() to handle zero pivots and regenerate matrices

## 11. Audit log (LLM calls)

**Total LLM calls:** 7

| # | Phase           | Provider      | Model    | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|----------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | qwen3:8b | 0         | 0   | 38168        |
| 2 | preregistration | ollama_remote | qwen3:8b | 0         | 0   | 26337        |
| 3 | novelty         | ollama_remote | qwen3:8b | 0         | 0   | 30443        |

| # | Phase    | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|----------|---------------|------------------|-----------|-----|--------------|
| 4 | novelty  | ollama_remote | qwen3:8b         | 0         | 0   | 20837        |
| 5 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15702        |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 14579        |
| 7 | judge    | ollama_remote | qwen3:8b         | 0         | 0   | 21453        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 167519 ms total latency. Provider mix: {'ollama\_remote': 7}

*(full prompt+response transcripts available in research/audit/d2ba6b1a27c2.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/d2ba6b1a27c2.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/d2ba6b1a27c2.tar.gz (if generated)